Esercitazione HTML, CSS e JavaScript con Media Query

Slide 1: Introduzione all'Esercitazione

- **Obiettivo**: L'obiettivo di questa esercitazione è creare una pagina web con l'uso di **HTML**, **CSS**, e **JavaScript**.
- **Strumenti**:
  - **HTML** per creare la struttura della pagina web.
  - **CSS** per applicare stili e layout.
  - **JavaScript** per rendere la pagina interattiva (ad esempio, con un quiz).
- **Risultato Atteso**: Una pagina web contenente testo, immagini, link e un quiz interattivo.

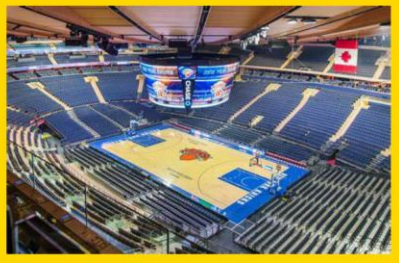
Basketball

Il basket è uno sport di squadra in cui due squadre cercano di segnare punti lanciando la palla in un canestro.

Regole principali:

- Una partita è divisa in 4 quarti da 10 o 12 minuti.
- Ogni squadra ha 5 giocatori in campo.
- Non è permesso camminare con la palla senza palleggiare.

Dove si gioca a Basket

New York	Los Angeles	Chicago
		
La città dell'NBA, con il famoso Madison Square Garden.	Casa dei Lakers e dei Clippers, icona del basket professionistico.	Dove Michael Jordan ha scritto la storia con i Bulls.

Per saperne di più, visita il sito ufficiale della NBA.

Quiz sul Basket

1. Quanti giocatori ha una squadra in campo?

- ○ 4
- ◉ 5
- ○ 6

2. Quali di questi elementi fanno parte del basket?

- ☑ Palla
- ☑ Tabellone
- ☐ Mazza

3. Quale città è famosa per i Bulls?

- ○ New York
- ◉ Chicago
- ○ Los Angeles

Verifica Risposte

Complimenti! Hai risposto correttamente a tutte le domande.

Cos'è HTML e la sua Struttura di Base

- **HTML (Hypertext Markup Language)** è il linguaggio usato per creare il contenuto di una pagina web.
- La struttura base di un file HTML include:
  - **`<html>`**: il tag radice che racchiude tutto il contenuto della pagina.
  - **`<head>`**: contiene informazioni sulla pagina, come il titolo e il collegamento ai file CSS o JavaScript.
  - **`<body>`**: contiene il contenuto visibile della pagina web (testo, immagini, link, ecc.).

Esempio di struttura di base:

```
<!DOCTYPE html>
<html lang="it">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Il mio sito</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>Benvenuto!</h1>
    <p>Questa è una pagina di esempio.</p>
</body>
</html>
```

- **Dichiarazione DOCTYPE**: Serve per specificare il tipo di documento. Nel caso di una pagina web moderna, si usa sempre `<!DOCTYPE html>`.
- **`<meta charset="UTF-8">`**: Imposta la codifica dei caratteri per supportare tutti i simboli e le lingue.
- **`<meta name="viewport" content="width=device-width, initial-scale=1.0">`**: Rende la pagina ottimizzata per dispositivi mobili, adattando la larghezza allo schermo e evitando lo zoom iniziale.

Organizzare i Contenuti con `<div>`, Classi e ID

- **`<div>`** è un elemento HTML usato per raggruppare altri elementi e applicare stili o layout.
- **ID e Classi**: Sono usati per selezionare e stilizzare specifici elementi.
  - Un **ID** è unico, quindi può essere utilizzato una sola volta nella pagina.
  - Una **Classe** può essere applicata a più elementi.

Esempio:

```
<div class="contenitore">
    <div id="box1">Contenuto 1</div>
    <div id="box2">Contenuto 2</div>
</div>
```

- **Differenza tra Classi e ID**:
    - **ID** è un identificatore unico: `id="box1"`
    - **Classi** sono generiche e riutilizzabili: `class="contenitore"`

Flexbox - Organizzare gli Elementi in Layout Flessibili

- **Flexbox** è una tecnica CSS moderna che consente di disporre gli elementi in modo flessibile. È particolarmente utile per costruire layout complessi e responsive.

  Proprietà principali di Flexbox:

    - **`display: flex`**: trasforma un contenitore in un contenitore flessibile.
    - **`justify-content`**: allinea gli elementi lungo l'asse principale (orizzontale per default).
    - **`align-items`**: allinea gli elementi lungo l'asse trasversale (verticale per default).
    - **`flex-direction`**: definisce la direzione in cui gli elementi devono essere disposti (riga, colonna, ecc.).

Esempio di Flexbox per allineare 3 elementi:

CSS:

```
.contenitore-flex {
    display: flex;
    justify-content: space-between;
}
.box {
    width: 30%;
    padding: 20px;
    background-color: lightblue;
    text-align: center;
}
```

HTML:

```
<div class="contenitore-flex">
    <div class="box">Elemento 1</div>
    <div class="box">Elemento 2</div>
    <div class="box">Elemento 3</div>
</div>
```

- **Flexbox vs. Float**:
    - **Flexbox** è più potente e flessibile rispetto all'uso di `float` per la disposizione degli elementi.
    - **Float** era usato in passato per creare layout, ma oggi **Flexbox** è la soluzione preferita.

Media Query - Adattare il Layout ai Dispositivi

- Le **media query** sono utilizzate per rendere il sito responsivo, adattandolo automaticamente a diversi dispositivi come smartphone, tablet e desktop.

  Cos'è una media query?

    - Una **media query** è una condizione che applica regole CSS solo se certe condizioni (come la larghezza del dispositivo) sono soddisfatte.
    - È fondamentale per la creazione di **siti web responsivi**, che si adattano a diverse risoluzioni di schermo.

Esempio di **media query**:

```
@media (max-width: 800px) {
    .bandiera {
        flex-direction: column;
    }
    .bandiera div {
        width: 100%;
    }
}
```

- **Spiegazione**:
    - La **media query** controlla la larghezza della finestra del browser.
    - Se la larghezza è inferiore o uguale a 800px, il layout cambia, passando da una disposizione orizzontale (in riga) a una verticale (in colonna).
- **Altri Esempi** di Media Query:
    - **Per schermi piccoli**:

      ```
      @media (max-width: 600px) {
          body {
              font-size: 14px;
          }
      }
      ```

    - **Per schermi grandi**:

```
@media (min-width: 1200px) {
    .contenitore {
        width: 80%;
    }
}
```

- Le media query sono fondamentali per creare siti **mobile-friendly**.

L'uso delle Immagini e dei Link

- **Immagini**: Il tag `<img>` viene utilizzato per inserire immagini. Puoi definire le dimensioni dell'immagine con l'attributo `width` e `height`, oppure tramite CSS.

```
<img src="img/basket.jpg" alt="Basket" width="300">
```

- **Link**: I link sono creati con il tag `<a>` e possono indirizzare a pagine interne o esterne.

```
<a href="https://www.nba.com">Visita il sito NBA</a>
```

- Per le immagini responsive:

```
img {
    width: 100%;
    height: auto;
}
```

Creare un Quiz Interattivo con JavaScript

- **JavaScript** viene usato per aggiungere interattività alla pagina. Ad esempio, possiamo creare un quiz che verifica le risposte degli utenti.

  Esempio di form HTML con radio button e checkbox:

```
<form onsubmit="return checkAnswers(event)">
  <p>1. Quanti giocatori ha una squadra di basket?</p>
  <input type="radio" name="giocatori" value="5"> 5
  <input type="radio" name="giocatori" value="4"> 4
  <input type="radio" name="giocatori" value="6"> 6
  <p>2. Quali di questi elementi fanno parte del basket?</p>
  <input type="checkbox" name="elementi" value="palla"> Palla
  <input type="checkbox" name="elementi" value="rete"> Rete
  <input type="checkbox" name="elementi" value="mazza"> Mazza
  <br><input type="submit" value="Verifica Risposte">
</form>
```

- **Funzionamento**: JavaScript verifica le risposte al quiz e restituisce il punteggio o un messaggio di feedback.

Introduzione a JavaScript

- **JavaScript (JS)** è un linguaggio di programmazione che permette di aggiungere interattività, dinamismo e logica alla tua pagina web.
- A differenza di **HTML** (struttura) e **CSS** (stile), **JavaScript** ti consente di:
  - Modificare dinamicamente il contenuto della pagina.
  - Rispondere alle azioni dell'utente (clic, invio di un form, ecc.).
  - Eseguire calcoli o logica per prendere decisioni.

Esempio di **JavaScript** per modificare un elemento della pagina:

```
document.getElementById('esempio').innerHTML = 'Ciao, mondo!';
```

Slide 10: Struttura di un File JavaScript

- **Inclusione di JavaScript nella pagina**:
  - Puoi inserire il codice JavaScript direttamente all'interno del file HTML con il tag `<script>`.
  - È possibile anche utilizzare file JavaScript separati e includerli con `<script src="script.js"></script>`.

Esempio di inclusione di un file JavaScript:

```
<!DOCTYPE html>
<html lang="it">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Pagina con JS</title>
    <script src="script.js"></script>
</head>
<body>
    <button onclick="cambiaTesto()">Clicca per cambiare il testo</button>
    <p id="esempio">Testo iniziale.</p>
</body>
</html>
```

Slide 11: Interagire con gli Elementi della Pagina

- **Selezionare elementi con JavaScript**:
  - **`getElementById`**: seleziona un elemento utilizzando l'ID.
  - **`querySelector`**: seleziona il primo elemento che corrisponde a un selettore CSS.

- **`querySelectorAll`**: seleziona tutti gli elementi che corrispondono a un selettore CSS.

Esempio:

```
let paragrafo = document.getElementById('esempio');
paragrafo.innerHTML = 'Nuovo testo!';
```

- **Manipolare il contenuto**: Puoi modificare il testo, cambiare il colore o altre proprietà CSS, aggiungere o rimuovere classi.

Gestire gli Eventi con JavaScript

- **Eventi**: JavaScript consente di rispondere agli eventi generati dagli utenti, come il clic su un pulsante, la selezione di una casella di controllo, o l'invio di un modulo.

Esempio di evento "click" su un pulsante:

```
<button onclick="cambiaTesto()">Clicca qui</button>
<script>
  function cambiaTesto() {
      document.getElementById('esempio').innerHTML = 'Testo modificato!';
  }
</script>
```

Slide 13: Creare un Quiz Interattivo con JavaScript

- **Obiettivo**: Creare un quiz in cui gli studenti possono rispondere a domande, e JavaScript verifica le risposte e restituisce un punteggio.

- La pagina contiene un modulo con radio button, checkbox, e un pulsante di invio. Quando l'utente invia il modulo, **JavaScript** calcola il punteggio e visualizza un messaggio di feedback.

Struttura HTML del Quiz

```
<form onsubmit="checkAnswers(event)">
    <p>1. Quanti giocatori ha una squadra di basket?</p>
    <input type="radio" name="giocatori" value="5"> 5
    <input type="radio" name="giocatori" value="4"> 4
    <input type="radio" name="giocatori" value="6"> 6
    <p>2. Quali di questi elementi fanno parte del basket?</p>
    <input type="checkbox" name="elementi" value="palla"> Palla
    <input type="checkbox" name="elementi" value="rete"> Rete
    <input type="checkbox" name="elementi" value="mazza"> Mazza
    <br><input type="submit" value="Verifica Risposte">
</form>
<p id="risultato" style="font-weight: bold; margin-top: 10px;"></p>
```

JavaScript per Verifica Risposte

```
function checkAnswers(event) {

    // Preveniamo il comportamento predefinito del form (pagina che si ricarica)
    event.preventDefault();

    let corrette = 0;
    let totalQuestions = 3;
    let resultText = "";
    // Controllo risposta numero 1
    let risposta1 = document.querySelector('input[name="giocatori"]:checked');
    if (risposta1 && risposta1.value === "5") {
        corrette++;
    }
    // Controllo risposta numero 2
    let risposta2 = document.querySelectorAll('input[name="elementi"]:checked');
    let risposta2Valori = Array.from(risposta2).map(el => el.value);
    if (risposta2Valori.includes("palla") && risposta2Valori.includes("tabellone")
&& !risposta2Valori.includes("mazza")) {
        corrette++;
    }
    // Controllo risposta numero 3
    let risposta3 = document.querySelector('input[name="citta"]:checked');
    if (risposta3 && risposta3.value === "Chicago") {
        corrette++;
    }
    // Mostra il risultato
    if (corrette === totalQuestions) {
        resultText = "Complimenti! Hai risposto correttamente a tutte le
domande.";
    } else {
        resultText = "Hai risposto correttamente a ${corrette} domande su
${totalQuestions}. Riprova!";
    }
    document.getElementById("risultato").innerText = resultText;
}
```

Spiegazione Passo per Passo

1. Prevenire il Comportamento Predefinito

```
event.preventDefault();
```

Quando invii un modulo (form), la pagina si ricarica per impostazione predefinita. La funzione preventDefault() impedisce questo comportamento, così che possiamo gestire il modulo tramite JavaScript senza ricaricare la pagina.

2. Controllare la Prima Domanda (radio button)

```
let risposta1 = document.querySelector('input[name="giocatori"]:checked');
```

Qui, stiamo selezionando l'input radio che l'utente ha scelto per la domanda "Quanti giocatori ci sono in una squadra di basket?".

- **querySelector('input[name="giocatori"]:checked')**: seleziona l'elemento input con il nome giocatori che è stato selezionato (cioè che ha l'attributo checked).

Dopo aver ottenuto la risposta dell'utente, confrontiamo il valore con la risposta corretta:

```
if (risposta1 && risposta1.value === "5") {
  corrette++;
}
```

Se la risposta dell'utente è corretta (cioè se il valore selezionato è "5"), incrementiamo il punteggio.

3. Controllare la Seconda Domanda (checkbox)

```
let risposta2 = document.querySelectorAll('input[name="sport"]:checked');
let risposta2Valori = Array.from(risposta2).map(el => el.value);
```

Per la seconda domanda, stiamo selezionando tutte le caselle di controllo (checkbox) che sono state selezionate. La funzione querySelectorAll() ci dà un elenco di tutte le risposte selezionate per quella domanda. Poi, con Array.from(), trasformiamo questa lista in un array e con map() otteniamo solo i valori delle risposte selezionate.

Adesso controlliamo che le risposte corrette siano selezionate e quelle sbagliate non lo siano:

```
if (risposta2Valori.includes("basket") && risposta2Valori.includes("calcio") &&
!risposta2Valori.includes("ping pong")) {
  corrette++;
}
```

Se l'utente ha selezionato "basket" e "calcio" e non ha selezionato "ping pong", incrementiamo il punteggio.

4. Mostrare il Risultato

Infine, mostriamo il risultato in un paragrafo con l'id risultato:

```
document.getElementById("risultato").innerText = risultatoText;
```

Questo aggiorna il contenuto del paragrafo con il messaggio di risultato, che può dire qualcosa come "Hai risposto correttamente a 2 domande su 2" o "Riprova!".

Passo 4: Collegare la Funzione al Form

Ora, dobbiamo collegare la funzione checkAnswers al modulo, così che venga eseguita quando l'utente invia il modulo:

```
<form id="quizForm" onsubmit="checkAnswers(event)">
```

La funzione checkAnswers(event) sarà chiamata quando l'utente invia il modulo, e il parametro event è passato per impedire che la pagina si ricarichi.

Esegui Azioni Condizionate in Base alla Risposta

- Puoi anche aggiungere logica condizionale per eseguire azioni diverse in base alla risposta dell'utente. Ad esempio:
    - **Se la risposta è corretta**, mostra un messaggio di congratulazioni.
    - **Se la risposta è sbagliata**, offri suggerimenti o incoraggiali a riprovare.

Esempio:

```
if (risposta1 && risposta1.value === "5") {
    alert("Risposta corretta!");
} else {
    alert("Risposta sbagliata, prova ancora!");
}
```